

Билет 9. Реализация `condition_variable` через `futex`. Счётчик поколений. Проблема `spurious wakeups`

0. Формулировка билета

Реализация `condition_variable` через `futex`. Счётчик поколений. Проблема `spurious wakeups`.

Главная идея билета:

`condition_variable` позволяет потоку уснуть до уведомления об изменении условия.

Но `futex` умеет ждать только значение одной ячейки памяти. Поэтому реализация `condition_variable` через `futex` использует отдельный атомарный счётчик поколений: каждое уведомление увеличивает счётчик, а ожидающий поток спит, пока счётчик не изменился.

Ключевая схема:

```
wait:
    s = seq.load()           // запомнить поколение
    unlock(mutex)
    futex_wait(&seq, s)      // спать, пока seq == s
    lock(mutex)

notify:
    seq.fetch_add(1)         // новое поколение
    futex_wake(...)
```

1. Что мы уже знаем про `condition_variable`

В билете 4 мы использовали `condition_variable` так:

```
std::mutex m;
std::condition_variable cv;
std::queue<Task> tasks;

void worker() {
    std::unique_lock<std::mutex> lock(m);

    cv.wait(lock, [&] {
```

```

        return !tasks.empty();
    });

    Task task = std::move(tasks.front());
    tasks.pop();
}

```

Пользовательское правило:

```
cv.wait(lock, pred);
```

эквивалентно:

```

while (!pred()) {
    cv.wait(lock);
}

```

Почему нужен `while`?

```

spurious wakeup
notify_all разбудил слишком много потоков
другой поток успел забрать ресурс
futex/syscall мог вернуться по сигналу

```

`condition_variable` не хранит само условие. Условие хранится во внешнем состоянии под `mutex`-ом:

```

!queue.empty()
count > 0
stop || !tasks.empty()
ready == true

```

2. Что умеет `futex`

`futex` умеет две базовые операции:

```

FUTEX_WAIT(addr, expected)
FUTEX_WAKE(addr, n)

```

`FUTEX_WAIT(addr, expected)`:

```
если *addr == expected:  
    поставить поток в очередь ожидания этого адреса  
    усыпить поток
```

```
если *addr != expected:  
    сразу вернуться, не засыпая
```

```
FUTEX_WAKE(addr, n):
```

```
разбудить до n потоков,  
которые ждут на этом адресе
```

То есть `futex` ждёт изменение **одной ячейки памяти**.

Но `condition_variable` должна ждать не одну ячейку, а произвольный предикат:

```
!q.empty()
```

или:

```
stop || !tasks.empty()
```

Поэтому нужен дополнительный механизм.

3. Почему нельзя просто ждать на предикате

Допустим, у нас есть очередь:

```
std::queue<int> q;  
Mutex3 m;
```

Предикат:

```
!q.empty()
```

Но `futex_wait` не может ждать “пока очередь станет непустой”.

Он принимает адрес и ожидаемое значение:

```
futex_wait(&some_int, expected);
```

Поэтому нельзя напрямую написать:

```
futex_wait(&q.empty(), true); // так нельзя
```

Нужна отдельная переменная, которая будет меняться при уведомлениях.

Так появляется:

```
std::atomic<uint32_t> seq{0};
```

`seq` — счётчик поколений.

4. Счётчик поколений

Счётчик поколений — это атомарное число, которое увеличивается при каждом уведомлении.

```
std::atomic<uint32_t> seq{0};
```

Смысл:

```
seq == 0 — уведомлений ещё не было
seq == 1 — было одно уведомление
seq == 2 — было два уведомления
...
```

Точнее:

`seq` не обязан означать точное число “полезных” уведомлений. Его роль — различать поколения ожидания: если `seq` изменился, значит, возможно, состояние поменялось, и нужно перепроверить предикат.

Важно:

```
condition_variable ждёт не сам pred,
а изменение поколения seq
```

Пользователь всё равно должен после пробуждения проверить предикат.

5. Базовая реализация `condition_variable` через `futex`

Упрощённый код:

```

class cond_var {
private:
    std::atomic<uint32_t> seq{0};

public:
    void wait(Mutex3& mtx) {
        uint32_t s = seq.load(std::memory_order_relaxed);

        mtx.unlock();

        futex_wait(&seq, s);

        mtx.lock();
    }

    void notify_one() {
        seq.fetch_add(1, std::memory_order_release);
        futex_wake(&seq, 1);
    }

    void notify_all() {
        seq.fetch_add(1, std::memory_order_release);
        futex_wake(&seq, INT_MAX);
    }
};

```

Это ровно базовая идея из конспекта: `wait` делает снимок поколения, отпускает внешний mutex, ждёт изменения `seq`, а после пробуждения снова захватывает mutex. `notify_one` / `notify_all` увеличивают поколение и будят ожидающих.

6. Очень важный момент: `mtx` внешний

`condition_variable` сама не защищает предикат.

Есть внешний mutex:

```

Mutex3 mtx;
std::queue<int> q;
cond_var cv;

```

Предикат:

```
!q.empty()
```

защищается именно `mtx`.

Ожидающий поток:

```
mtx.lock();

while (q.empty()) {
    cv.wait(mtx);
}

auto x = q.front();
q.pop();

mtx.unlock();
```

Уведомляющий поток:

```
mtx.lock();

q.push(42);

mtx.unlock();

cv.notify_one();
```

Идея:

```
mtx защищает q
cv/seq отвечает только за сон и пробуждение
```

7. Что делает `wait`

```
void wait(Mutex3& mtx) {
    uint32_t s = seq.load();

    mtx.unlock();

    futex_wait(&seq, s);

    mtx.lock();
}
```

По шагам:

1. Поток входит в `wait`, уже держа внешний `mutex`.
2. Он читает `seq` и запоминает снимок поколения `s`.

3. Он отпускает внешний mutex.
4. Он вызывает `futex_wait(&seq, s)`.
5. Если `seq` всё ещё равен `s`, поток засыпает.
6. Если `seq` уже изменился, поток не засыпает и сразу возвращается.
7. После пробуждения поток снова захватывает внешний mutex.
8. После возврата из `wait` пользовательский код перепроверяет предикат.

Главная аналогия с обычным `condition_variable`:

```
wait временно отпускает mutex
пока поток спит, другие могут изменить состояние
перед возвратом wait снова захватывает mutex
```

8. Почему снимок `seq` берётся до `unlock`

Это ключевой момент билета.

Правильно:

```
uint32_t s = seq.load();
mtx.unlock();
futex_wait(&seq, s);
```

Снимок поколения берётся **пока поток ещё держит mutex**.

Почему?

Пока поток держит mutex, состояние-предикат стабильно: другой поток не может одновременно изменить очередь/счётчик/флаг, защищённые этим mutex-ом.

Плохой вариант:

```
mtx.unlock();
uint32_t s = seq.load();
futex_wait(&seq, s);
```

Тогда возможен `lost wakeup`.

Сценарий:

```
thread A:
    проверил pred == false
    собирается ждать
    unlock(mtx)
```

```

thread B:
    lock(mtx)
    меняет pred на true
    unlock(mtx)
    notify_one(): seq++

thread A:
    только теперь делает s = seq.load()
    запоминает уже новое поколение
    futex_wait(&seq, s)
    засыпает, хотя уведомление уже произошло

```

В этом случае поток A может уснуть навсегда.

Поэтому снимок `seq` должен быть сделан **до отпускания mutex-а**.

9. Как поколение устраняет lost wakeup

Правильный порядок:

```

uint32_t s = seq.load(); // пока держим mtx
mtx.unlock();
futex_wait(&seq, s);

```

Теперь рассмотрим опасный сценарий:

```

thread A:
    s = seq.load() // пусть s = 10
    mtx.unlock()

thread B:
    lock(mtx)
    меняет pred
    unlock(mtx)
    notify_one():
        seq становится 11
        futex_wake

thread A:
    futex_wait(&seq, 10)

```

Что делает `futex_wait`?

Он проверит:


```
seq == 10 ?
```

Но `seq` уже `11`.

Значит:

```
futex_wait сразу вернётся  
thread A не заснёт
```

Уведомление не потерялось. Оно превратилось в факт:

```
поколение уже изменилось
```

Именно это и есть смысл generation counter.

В конспекте это формулируется так: поколение превращает “потерянный сигнал” в “значение уже изменилось”.

10. Почему `notify` увеличивает `seq`

`notify_one`:

```
void notify_one() {  
    seq.fetch_add(1);  
    futex_wake(&seq, 1);  
}
```

Сначала меняем поколение:

```
seq = seq + 1
```

Потом будим одного ожидающего.

Почему важно менять `seq`?

Потому что ожидающие потоки спят так:

```
futex_wait(&seq, old_seq);
```

Если `seq` изменился, они больше не должны продолжать спать в старом поколении.

Если поток ещё не успел заснуть, `futex_wait` увидит новое значение и не уснёт.

11. Почему `notify` не обязан знать предикат

`condition_variable` сама не знает, что именно ждут потоки.

Один поток может ждать:

```
!q.empty()
```

другой — теоретически другой предикат на том же состоянии.

`notify_one` просто говорит:

```
что-то могло измениться  
проснись и перепроверь
```

Поэтому `seq` — грубый сигнал:

```
было уведомление  
возможно, предикат стал истинным
```

Но он не гарантирует:

```
твой конкретный предикат точно истинный
```

12. Почему остаются *spurious wakeups*

Даже с `seq` и `futex` пробуждение не означает, что предикат истинный.

Причины:

1. Другой поток успел забрать ресурс

Пример с очередью:

```
q была пустая  
producer положил одну задачу  
notify_one разбудил consumer A  
  
но consumer B успел захватить mutex раньше  
B забрал задачу
```

```
A наконец захватил mutex  
q снова пустая
```

Consumer A проснулся, но его предикат:

```
!q.empty()
```

снова ложен.

Поэтому нужен цикл.

2. `notify_all`

`notify_all` будит всех.

```
producer положил одну задачу  
notify_all разбудил 10 потоков  
один забрал задачу  
остальные проснулись зря
```

Остальные должны проверить предикат и снова уснуть.

3. `futex_wait` может вернуться по причинам ОС

Например:

```
сигнал  
EINTR  
spurious wakeup на уровне реализации
```

Опять же: после возврата из `wait` нужно проверить предикат.

13. Почему `while`, а не `if`

Плохо:

```
mtx.lock();  
  
if (q.empty()) {  
    cv.wait(mtx);  
}
```

```
auto x = q.front(); // может быть ошибка
q.pop();

mtx.unlock();
```

Почему плохо?

После `cv.wait` очередь всё ещё может быть пустой.

Правильно:

```
mtx.lock();

while (q.empty()) {
    cv.wait(mtx);
}

auto x = q.front();
q.pop();

mtx.unlock();
```

Или в стиле стандартной библиотеки:

```
std::unique_lock<std::mutex> lock(m);
cv.wait(lock, [&] {
    return !q.empty();
});
```

Ключевая фраза:

`notify` означает “возможно, состояние изменилось”, а не “условие точно выполнено”.

14. Почему spurious wakeup здесь не дефект

В конспекте подчёркивается: spurious wakeup — не дефект реализации, а прямое следствие конструкции на поколениях. `seq` — грубый сигнал “было хоть одно уведомление”, но не привязка к конкретному предикату.

Поэтому `condition_variable` специально имеет такую семантику:

```
wait может вернуться без истинного pred
пользователь обязан проверить pred сам
```

Это делает реализацию простой, эффективной и универсальной.

15. `notify_one`

```
void notify_one() {  
    seq.fetch_add(1);  
    futex_wake(&seq, 1);  
}
```

Смысл:

сменить поколение
разбудить одного ожидающего

Когда использовать:

добавили одну задачу в очередь
освободили один ресурс
семафор release на 1

Но важно:

`notify_one` не передаёт ресурс напрямую конкретному потоку. Он только будит одного ждущего. После пробуждения поток всё равно должен захватить mutex и проверить предикат.

16. `notify_all`

```
void notify_all() {  
    seq.fetch_add(1);  
    futex_wake(&seq, INT_MAX);  
}
```

Смысл:

сменить поколение
разбудить всех ожидающих

Когда использовать:

```
stop = true
закрытие очереди
барьер открылся
состояние изменилось так, что многим потокам пора перепроверить pred
```

Минус:

thundering herd

Все просыпаются, конкурируют за mutex, многие снова засыпают.

17. Оптимизация: FUTEX_REQUEUE

Наивный `notify_all`:

```
futex_wake(&seq, INT_MAX);
```

будит всех ожидающих.

Проблема:

```
все проснулись
все пытаются взять внешний mutex
mutex может взять только один
остальные снова блокируются
```

Это thundering herd.

Оптимизация:

FUTEX_REQUEUE

Идея:

```
разбудить одного ожидающего
остальных перевесить из очереди ожидания condition_variable
в очередь ожидания связанного mutex-а
```

Тогда потоки будут просыпаться по очереди по мере освобождения mutex-а.

В конспекте это описано как более правильный способ вместо `futex_wake(INT_MAX)` для `notify_all`: разбудить одного, а остальных requeue-нуть на mutex, чтобы избежать шторма.

Для экзамена достаточно сказать:

Простая реализация `notify_all` через `futex_wake(INT_MAX)` понятна, но может вызвать thundering herd. Реальные реализации могут использовать `FUTEX_QUEUEUE`, чтобы перевесить ожидающих с CV на mutex.

18. Связь с `std::atomic::wait`

В C++20 есть:

```
std::atomic<int> a{0};

a.wait(0);

// другой поток
a.store(1);
a.notify_one();
```

Это похоже на futex-подобную парковку прямо на атомике.

Связь:

futex – Linux syscall для ожидания по адресу
atomic::wait – стандартный C++ интерфейс похожей идеи
condition_variable через futex – более высокоуровневая обёртка со внешним mutex и pred

В конспекте по CV отдельно отмечено, что `std::atomic::wait/notify` — низкоуровневый аналог condition variable на атомике, а в Linux под капотом может использоваться futex.

19. Упрощённый код с `wait(lock, pred)`

Пользовательский интерфейс обычно выглядит так:

```
class cond_var {
private:
    std::atomic<uint32_t> seq{0};

public:
    void wait(Mutex3& mtx) {
        uint32_t s = seq.load(std::memory_order_relaxed);
        mtx.unlock();
        futex_wait(&seq, s);
        mtx.lock();
    }
};
```

```

    }

    template <typename Pred>
    void wait(Mutex3& mtx, Pred pred) {
        while (!pred()) {
            wait(mtx);
        }
    }

    void notify_one() {
        seq.fetch_add(1, std::memory_order_release);
        futex_wake(&seq, 1);
    }

    void notify_all() {
        seq.fetch_add(1, std::memory_order_release);
        futex_wake(&seq, INT_MAX);
    }
};

```

Использование:

```

Mutex3 m;
cond_var cv;
std::queue<int> q;

void popper() {
    m.lock();

    cv.wait(m, [&] {
        return !q.empty();
    });

    int x = q.front();
    q.pop();

    m.unlock();
}

void pusher() {
    m.lock();

    q.push(42);

    m.unlock();

    cv.notify_one();
}

```


20. Важные ограничения учебной реализации

Эта реализация показывает идею, но реальные `condition_variable` сложнее.

В реальных реализациях могут быть:

```
отдельные счётчики waiters
рекеув на mutex
особые правила memory ordering
обработка timeout
обработка cancellation/signals
оптимизации notify_one/notify_all
разные очереди ожидания
```

Но для экзамена центральная идея:

```
generation counter решает lost wakeup
spurious wakeups остаются
поэтому нужен while
```

21. Короткий ответ на экзамене

Можно отвечать так:

`condition_variable` должна позволить потоку уснуть до уведомления, но `futex` умеет ждать только значение одной ячейки памяти. Поэтому реализация через `futex` использует счётчик поколений `seq`. `seq` — атомарное число, которое увеличивается при каждом `notify`.

В `wait` поток уже держит внешний `mutex`, который защищает предикат. Пока `mutex` ещё удерживается, поток делает снимок поколения: `s = seq.load()`. Затем он отпускает `mutex` и вызывает `futex_wait(&seq, s)`. Если между `unlock` и `futex_wait` другой поток успел сделать `notify` и увеличить `seq`, то `futex_wait` увидит, что `seq != s`, и сразу вернётся, не засыпая. Так счётчик поколений устраняет `lost wakeup`.

`notify_one` делает `seq.fetch_add(1)` и `futex_wake(&seq, 1)`. `notify_all` увеличивает `seq` и будит всех, например через `futex_wake(INT_MAX)`, хотя в реальных реализациях могут использовать `FUTEX_REQUEUE`, чтобы избежать `thundering herd`.

При этом `seq` не означает, что конкретный предикат точно стал истинным. Это только грубый сигнал “было уведомление”. Другой поток мог уже забрать ресурс, `notify_all` мог разбудить слишком много потоков, или `futex` мог вернуться по

сигналу. Поэтому после `wait` пользователь обязан перепроверить предикат в цикле `while (!pred) wait(...)`. Это и есть причина spurious wakeups.

22. Мини-код для ответа на доске

```
class cond_var {
private:
    std::atomic<uint32_t> seq{0};

public:
    void wait(Mutex3& mtx) {
        uint32_t s = seq.load(std::memory_order_relaxed);

        mtx.unlock();

        futex_wait(&seq, s);

        mtx.lock();
    }

    void notify_one() {
        seq.fetch_add(1, std::memory_order_release);
        futex_wake(&seq, 1);
    }

    void notify_all() {
        seq.fetch_add(1, std::memory_order_release);
        futex_wake(&seq, INT_MAX);
    }
};
```

Что объяснить:

```
seq – generation counter
s – снимок поколения
снимок берём до unlock
futex_wait(&seq, s) спит, пока seq == s
notify меняет seq
если notify случился до сна, seq уже другой, и wait не заснёт
после wait всё равно проверяем pred в while
```

23. Мини-проверка

1. Почему `condition_variable` нельзя реализовать просто ожиданием произвольного предиката через `futex`?

2. Что такое счётчик поколений `seq`?
3. Что делает `wait` в реализации CV через futex?
4. Почему снимок `seq` нужно брать до `mtx.unlock()`?
5. Как `seq` защищает от lost wakeup?
6. Что делает `notify_one`?
7. Что делает `notify_all`?
8. Почему после `wait` всё равно нужен `while`, а не `if`?
9. Почему spurious wakeup здесь считается нормальной частью дизайна?
10. Что такое thundering herd при `notify_all`?
11. Что делает `FUTEX_REQUEUE`?
12. Чем `condition_variable` отличается от простого `futex_wait` на одной переменной?